TEC | Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE 1110 – Análisis de Señales Mixtas

Taller 3 - Avance Proyecto

Sistema completo, emisor ESP32 y receptor Pico

Profesor:

Luis Alberto Chavarría Zamora

Estudiantes:

Nicolás Florez Jiménez - 2024086367

Tamara Vanessa Cajiao Molina - 2024143333

Santiago Robles Obando - 2022207100

Santiago Boza Quiros - 2024174347

II Semestre 2026

30 de Octubre

0. Índice

0. Índice 2
1. Introducción y objetivo 3
2. Arquitectura y criterios de diseño 3
2.1. Diagrama general del sistema completo 5
3. Diseño e implementación 5
3.1. Modelo en Python 5
3.1.1. Organización y ejecución 5
3.1.2. Señales prototipadas 7
3.1.3. Filtros y resultados simulados 7
3.2. Emisor en ESP32 10
3.2.1. Objetivo y funcionamiento 10
3.2.2. Señales y parámetros de trabajo 11
3.2.3. Preparación y reproducción de los datos 12
3.2.4. Conexiones y manejo de la demostración 13
3.3. Evaluación digital en ESP32 13
3.3.1. Organización del procesamiento 13
3.3.2. Filtro temporal y filtro espectral 14
3.3.3. Alineación y cálculo de métricas 14
3.3.4. Criterios de diseño y compromisos 15
3.4. Receptor y estrategia de validación 15
3.4.1. Conexión y acondicionamiento 15
3.4.2. Adquisición y muestreo 16
3.4.3. Procesamiento e identificación de frecuencias 17
3.4.4. Detección de problemas y criterio de decisión 18
4. Validación y resultados 20
4.1. Tabla de pruebas 20

4.2. Resultados del emisor y de la evaluación en ESP32 21
4.3. Comparación entre simulación y ESP32 23
4.4. Resumen de la evidencia 24
5. Discusión 24
6. Referencias 25

1. Introducción y objetivo

El Proyecto Grupal 1 consiste en diseñar un sistema distribuido de tres nodos que adquiera una señal contaminada, caracterice el ruido y lo reduzca en tiempo real mediante un filtro temporal y uno espectral, para después comparar ambas salidas con el error cuadrático medio (MSE), la relación señal a ruido (SNR), la energía conservada y la latencia [1].

Este avance responde a la sesión asincrónica de la semana 12 [2], que solicita tres elementos: (1) el sistema completo en alto nivel, con las frecuencias de trabajo prototipadas; (2) el emisor en el microcontrolador con frecuencias establecidas; y (3) diagramas con la estrategia de validación en el receptor del microcontrolador. Las secciones 3.1, 3.2 y 3.4 cubren respectivamente cada uno de ellos.

El documento distingue tres tipos de evidencia: simulación en computadora (prototipo en Python), ejecución en el microcontrolador (ESP32, con resultados de consola) y pruebas en computadora de los métodos del receptor en Raspberry Pi Pico.

La Sección 2 resume la arquitectura y los parámetros comunes. La Sección 3 describe el prototipo Python, el emisor y la evaluación digital en ESP32 y el receptor con su estrategia de validación. La Sección 4 presenta la tabla de validación y compara los resultados, y la Sección 5 discute los resultados, las decisiones de diseño y sus limitaciones.

2. Arquitectura y criterios de diseño

Los parámetros de las señales se acordaron una sola vez y se mantienen iguales en Python, en el firmware del emisor y en este documento (Tabla 2.1). Cambiar una señal exige regenerarla en Python, exportarla y volver a compilar el firmware, de modo que todas las plataformas trabajan siempre con las mismas muestras.

Parámetro	Valor	Criterio
Frecuencia de muestreo	16 000 Hz	Nyquist en 8 kHz, el doble de la interferencia más alta (4 kHz)
Bloque FFT	N = 1024 = 2^{10}	Δf = 15.625 Hz y 64 ms por bloque; potencia de dos para FFT radix 2
Receptor Pico	fs = 10 000 Hz, N = 256	Nyquist en 5 kHz; Δf = 39.06 Hz y 25.6 ms por bloque
Señal útil	440, 880 y 1320 Hz	Tres componentes, como exige el proyecto; amplitudes 1, 0.55 y 0.3
Ruido caso 1	Tono de 4000 Hz, SNR 10 dB	Interferencia sinusoidal separada
Ruido caso 2	Banda 2500–3800 Hz, SNR 5 dB	Ruido de alta frecuencia fuera de la banda útil
Ruido caso 3	Banda 600–1500 Hz, SNR 0 dB	Superposición parcial: compromiso ruido–distorsión
Duración	6 s = 96 000 muestras	Múltiplo de 256 (bloques DAC y FIR); 93 bloques de 1024
Escala de la mezcla	Pico ≈ 0.8	Códigos DAC 25–230 (≈ 0.32–2.98 V), sin recorte
Formato numérico	float64 / float32 / u8 / u12	Python / filtros ESP32 / DAC ESP32 / ADC Pico

Tabla 2.1. Configuración común del avance.

El muestreo de 16 kHz evita aliasing en todas las condiciones generadas, porque ninguna componente supera los 4 kHz [3]. Con N = 1024 los tonos útiles quedan separados por unos 28 bins, suficiente para distinguirlos aun con el lóbulo principal más ancho de la ventana Hann [4]. La longitud de bloque es potencia de dos, como pide el proyecto, y permite emplear la FFT radix 2 [5]. El receptor muestrea a 10 kHz con bloques de 256 (Sección 3.4): su Nyquist de 5 kHz también supera la componente más alta y los tonos útiles quedan separados por unos 11 bins.

2.1. Diagrama general del sistema completo

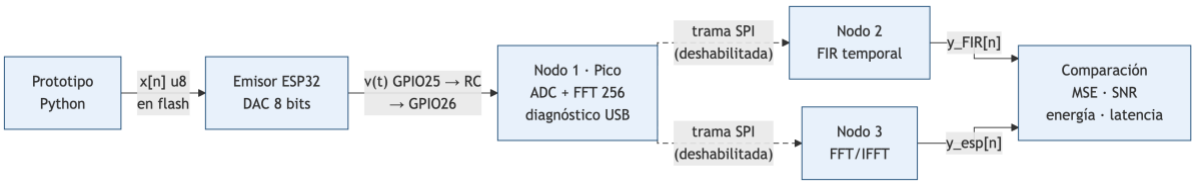


Figura 2.1. Sistema completo. Línea discontinua: enlace SPI entre nodos, deshabilitado en este avance.

La Figura 2.1 ubica cada parte del avance dentro de la arquitectura mínima del proyecto [1]. El prototipo Python genera los casos; el ESP32 los emite por su DAC y, en este avance, ejecuta además el filtro FIR y el espectral sobre las mismas muestras para compararlos en el microcontrolador. El nodo 1, en la Raspberry Pi Pico, adquiere la señal por su ADC, la analiza por bloques y reporta su diagnóstico por USB. Las líneas discontinuas representan el enlace SPI entre nodos: la Pico define la trama con delimitación de bloques y CRC-8, y en este avance se mantiene deshabilitada porque ambos filtros se evalúan dentro del ESP32.

3. Diseño e implementación

3.1. Modelo en Python

3.1.1. Organización y ejecución

El sistema completo en alto nivel se ejecuta con un único programa, proyecto_1_grupal/prototipo.py, sin notebooks ni pasos manuales [6]. A partir de las semillas y parámetros de configuraciones.json genera los tres casos, ejecuta un ejemplo de DFT directa frente a FFT, diseña y verifica el filtro FIR, compara candidatos IIR, aplica el filtro espectral, evalúa el compromiso entre ruido y distorsión y exporta figuras, audios WAV, muestras y tablas. Los módulos de procesamiento son los mismos de la Tarea 1 y emplean NumPy y SciPy [7], [8].

Los ajustes de procesamiento se leen de prototipo.json, por lo que pueden modificarse sin editar el código. Cada ejecución guarda la configuración efectiva (configuracion_ejecutada.json y configuracion.csv), de modo que cada resultado queda asociado a sus parámetros. La instalación y los comandos se documentan en el archivo README_EJECUCION.txt del paquete de entrega.

Bloque	Parámetro	Valor
FIR	Banda de paso / rechazo	0–1600 Hz / ≥ 2400 Hz
FIR	Rizado máximo / atenuación mínima	0.1 dB / 60 dB
FIR	Diseño	Kaiser, beta = 5.653, 75 coeficientes (orden 74)
FIR	Retardo de grupo / bloque	37 muestras (2.31 ms) / 256 muestras
Espectral	Ventana / bloque / salto	Hann / 1024 / 512 (50 % de solapamiento)
Espectral	Frecuencia mínima de selección / ancho	1600 Hz / 100 Hz
Espectral	Umbral relativo / factor de amplitud	0.05 / 0.1
Compromiso	Caso / frecuencia mínima / factores	caso_3 / 600 Hz / 0.8 y 0.1
Verificación	Tolerancia numérica	10^{-12}

Tabla 3.1. Parámetros de procesamiento del prototipo (prototipo.json).

El programa se detiene si una verificación falla: la DFT directa debe coincidir con la FFT (error máximo obtenido de 1.9×10^{-15}), las implementaciones propias del FIR deben coincidir con SciPy, el FIR debe cumplir rizado y atenuación y el filtro espectral con factor unitario debe reconstruir la entrada. En la ejecución del 30 de octubre de 2026 el programa terminó sin errores y sus métricas coincidieron con las de la Tarea 1 con diferencias menores que 10^{-9}.

3.1.2. Señales prototipadas

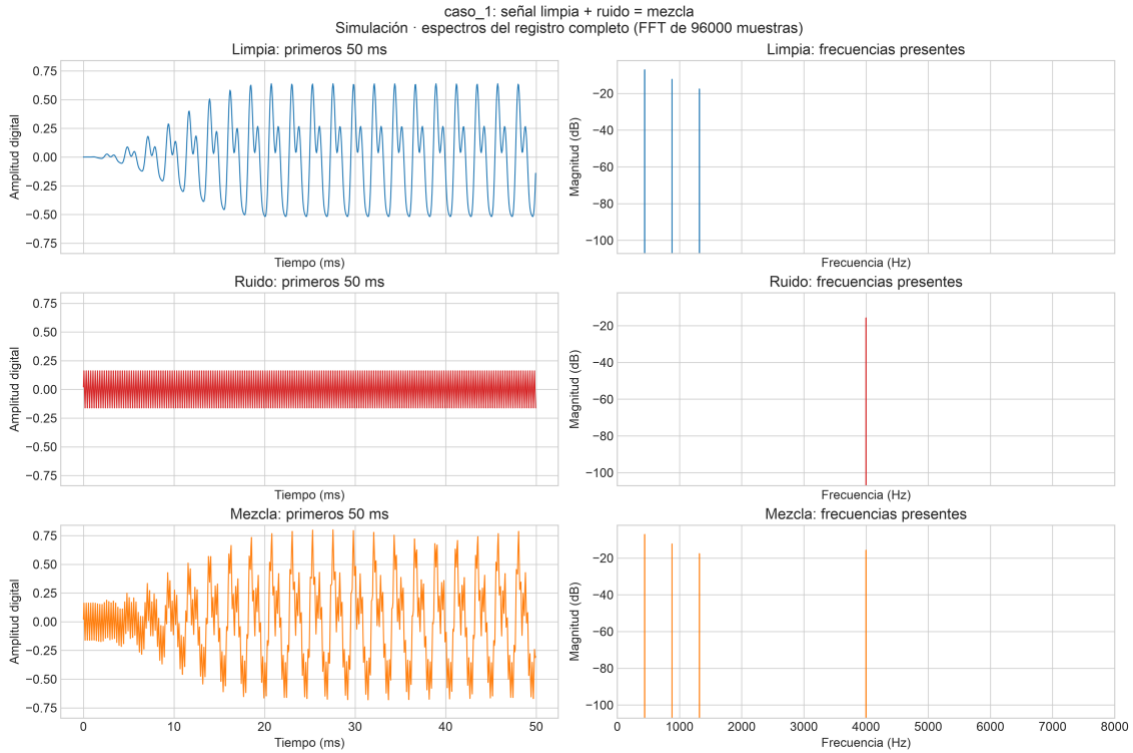


Figura 3.1. Caso 1 simulado: señal limpia, ruido y mezcla en tiempo y frecuencia.

La Figura 3.1 muestra el caso 1: la mezcla contiene los tres tonos útiles y la interferencia de 4000 Hz. El ruido del caso 2 ocupa 2500–3800 Hz y el del caso 3, 600–1500 Hz. Este último se superpone con la señal útil, como exige el enunciado para evidenciar el compromiso entre eliminar ruido y distorsionar la señal [1].

3.1.3. Filtros y resultados simulados

El FIR se diseñó con ventana de Kaiser para una banda de paso hasta 1600 Hz y una de rechazo desde 2400 Hz. Con 75 coeficientes alcanza una atenuación mínima de 60.4 dB y un rizado de 0.016 dB pico a pico, con retardo de grupo constante de 37 muestras (Figura 3.2). La Tabla 3.2 justifica su elección: los candidatos IIR cumplen la misma especificación con menos operaciones, pero su fase no lineal deforma la señal útil y reduce la SNR de salida [3].

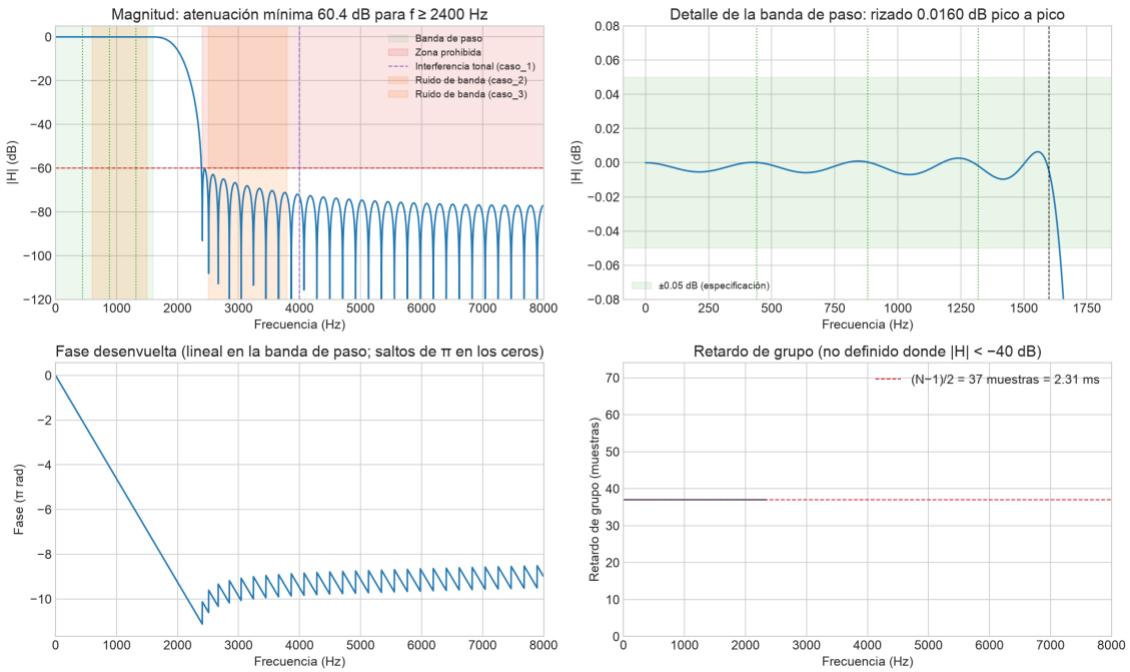


Figura 3.2. Respuesta en frecuencia, fase y retardo de grupo del FIR.

Filtro	Orden	MAC por muestra	Retardo de grupo (muestras)	SNR salida caso 1 (dB)
FIR Kaiser	74	75	37 (constante)	69.05
IIR Butterworth	20	50	18.4 – 24.7	15.46
IIR Chebyshev I	10	25	13.3 – 20.6	14.84
IIR Elíptico	6	15	6.0 – 10.6	18.64

Tabla 3.2. FIR frente a candidatos IIR de igual especificación (SNR con el mejor retardo entero).

El filtro espectral procesa bloques de 1024 muestras con ventana Hann y 50 % de solapamiento. Por encima de 1600 Hz selecciona las componentes que superan un umbral relativo de energía y las atenúa a 0.1 en amplitud, conservando la simetría conjugada del espectro. El umbral se recalcula en cada bloque, de modo que el método se adapta a la interferencia presente y no anula una frecuencia fija (Figura 3.3) [1].

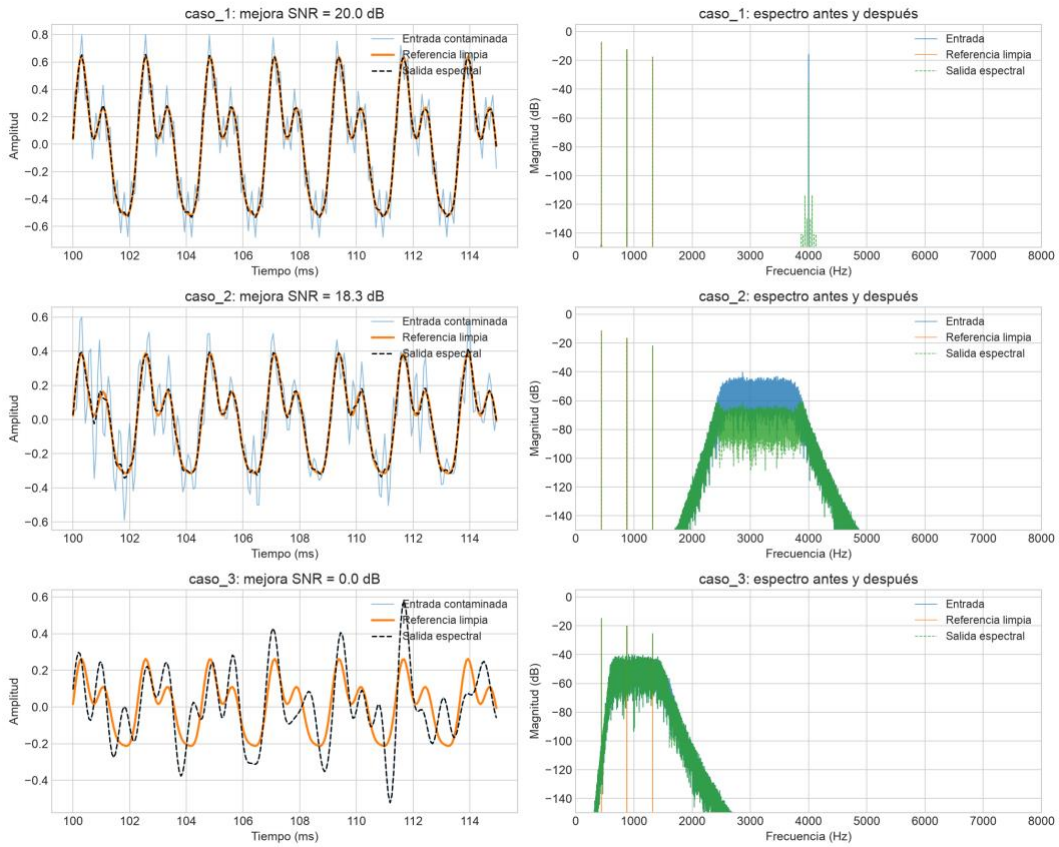


Figura 3.3. Salida del filtro espectral en los tres casos (tiempo y espectro).

Caso	Método	SNR entrada (dB)	SNR salida (dB)	ΔSNR (dB)	MSE	Energía útil (%)
caso_1	Sin filtrar	10.0017	10.0017	0.0000	1.329e-02	100.0000
caso_1	FIR	10.0017	69.0488	59.0471	1.655e-08	100.0013
caso_1	Espectral	10.0017	29.9760	19.9743	1.337e-04	100.0000
caso_2	Sin filtrar	5.0017	5.0017	0.0000	1.555e-02	100.0000
caso_2	FIR	5.0017	65.1397	60.1379	1.507e-08	100.0013
caso_2	Espectral	5.0017	23.3486	18.3468	2.276e-04	100.0000
caso_3	Sin filtrar	0.0005	0.0005	0.0000	2.241e-02	100.0000
caso_3	FIR	0.0005	0.0034	0.0029	2.239e-02	100.0013
caso_3	Espectral	0.0005	0.0027	0.0022	2.239e-02	100.0000

Tabla 3.3. Resultados simulados sobre 95 963 muestras alineadas por caso.

En los casos 1 y 2 ambos métodos mejoran la SNR. El FIR mejora más porque su banda de rechazo cubre todo el ruido, mientras que el espectral conserva el 10 % de la amplitud de las componentes seleccionadas. En el caso 3 ninguna configuración base mejora, porque el ruido cae dentro de la banda protegida. Al bajar la frecuencia mínima a 600 Hz, el factor 0.1 mejora la SNR en 5.90 dB pero conserva solo el 72.1 % de la energía útil, frente a 1.81 dB y 89.9 % con factor 0.8: es el compromiso entre ruido y distorsión que pide el proyecto. Los tiempos por bloque medidos en la computadora no representan los del microcontrolador y no se usan como evidencia de tiempo real.

3.2. Emisor en ESP32

3.2.1. Objetivo y funcionamiento

El emisor reproduce en un microcontrolador las señales utilizadas en el modelo Python del proyecto. Su propósito es disponer de una fuente con frecuencias y condiciones conocidas para estudiar la reducción de ruido. Este desarrollo corresponde al emisor con frecuencias establecidas solicitado en el enunciado del taller [2].

Se utilizó una ESP32 clásica, programada en lenguaje C con ESP-IDF 5.5.5. El programa reproduce tres casos mediante el convertidor digital a analógico (DAC), que transforma una secuencia de números en una salida de tensión.

Para cada caso se realizan tres etapas en orden: primero se reproduce la mezcla por el DAC, luego se evalúa el filtro temporal y finalmente el filtro espectral. Los filtros se calculan en la ESP32; la computadora prepara los datos, programa la placa y presenta la consola. La Figura 3.4 representa este recorrido y distingue la salida eléctrica de la evaluación digital.

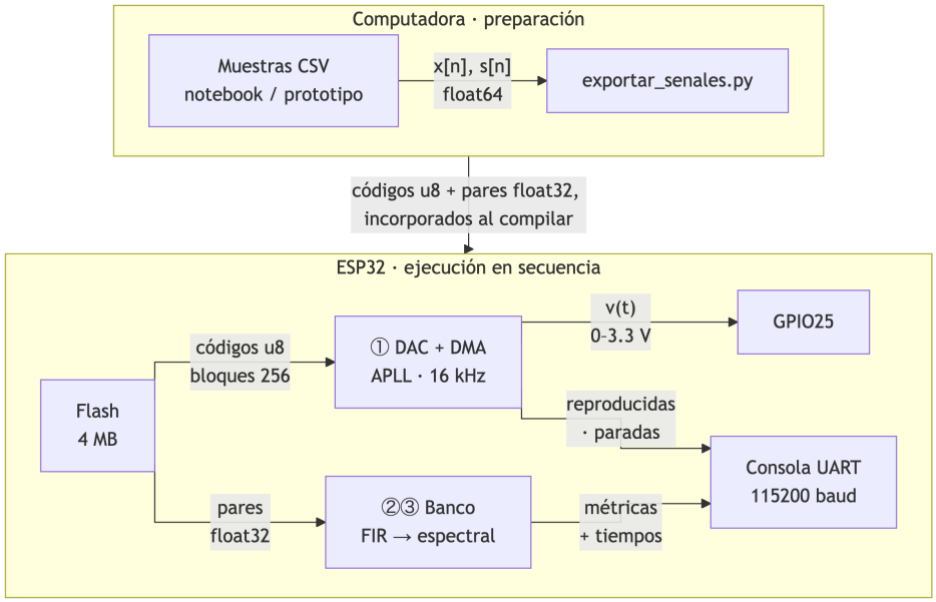


Figura 3.4. Flujo de preparación, emisión y evaluación de las señales en ESP32.

3.2.2. Señales y parámetros de trabajo

Se conservaron los casos y las semillas del modelo Python. Así se comparan las implementaciones de Python y C con las mismas señales, sin generar ruidos diferentes para cada plataforma. Las frecuencias, amplitudes y fases proceden de las configuraciones del modelo [6]. Las amplitudes de la Tabla 3.4 son las configuradas antes del escalado común que realiza el modelo.

Parámetro	Valor o configuración
Plataforma	ESP32 clásica; firmware C con ESP-IDF 5.5.5
Frecuencia de muestreo	16 000 Hz
Duración / muestras por caso	6 s / 96 000
Frecuencias útiles	440, 880 y 1320 Hz
Amplitudes configuradas	1, 0.55 y 0.3
Fases configuradas	0°, 35° y –50°
Salida analógica	GPIO25 (DAC canal 0), referencia en GND
Resolución del DAC	8 bits: códigos de 0 a 255
Memoria flash requerida	Al menos 4 MB

Tabla 3.4. Parámetros comunes de las señales y del emisor.

Caso	Ruido añadido	SNR configurada	Semilla
1	Interferencia tonal de 4000 Hz	10 dB	1000
2	Ruido de banda entre 2500 y 3800 Hz	5 dB	2000
3	Ruido entre 600 y 1500 Hz, superpuesto a la señal útil	0 dB	3000

Tabla 3.5. Casos reproducidos en el taller.

El muestreo de 16 000 Hz conserva la configuración del modelo y deja margen para la interferencia tonal más alta, de 4000 Hz. Los dos primeros casos permiten estudiar ruido separado de los tonos útiles; el tercero muestra qué ocurre cuando el ruido comparte su región de frecuencias.

3.2.3. Preparación y reproducción de los datos

Las muestras se generan en Python y se exportan como archivos CSV. Un exportador prepara dos representaciones: códigos enteros de 8 bits para el DAC y muestras en punto flotante de 32 bits para evaluar los filtros. Los datos se incorporan a la memoria flash al compilar el firmware, que usa una única partición de aplicación de casi 4 MB [9].

Para el DAC se conserva la escala del modelo, se limita la amplitud al intervalo [−1, 1] y cada muestra se transforma en un código entre 0 y 255. Los tres registros exportados reportaron cero muestras recortadas. Las métricas usan las muestras digitales de mezcla y referencia en su escala original; por ello, no incluyen el efecto de la conversión de 8 bits.

Guardar los registros evita que la computadora transmita muestras durante la demostración y garantiza que cada reinicio use los mismos datos. La contrapartida es el espacio ocupado en flash y la necesidad de exportar, compilar y programar de nuevo cuando se cambian las señales.

La reproducción usa acceso directo a memoria (DMA) para transferir los códigos al DAC por bloques, con el reloj APLL configurado a 16 kHz. Así se reduce el trabajo de alimentación del periférico y no se carga toda la señal en la memoria de trabajo de la ESP32 [10].

3.2.4. Conexiones y manejo de la demostración

La conexión USB permite alimentar la placa, cargar el firmware y observar los mensajes de ejecución. GPIO25 es la salida de señal del emisor y GND proporciona la referencia eléctrica común del punto de medición o del receptor.

El DAC entrega una señal unipolar: el cero de la señal numérica queda aproximadamente a media alimentación (≈ 1.65 V) y la mezcla ocupa ≈ 0.32–2.98 V. La placa no incluye un filtro de reconstrucción propio; esa función la cumple el filtro RC de la entrada del receptor (Sección 3.4).

Al arrancar, el firmware recorre automáticamente los tres casos. La consola muestra el inicio y el final de cada reproducción, los resultados de los filtros y un resumen comparativo; el botón EN/RESET repite la secuencia completa. Los comandos de compilación, programación y monitorización están documentados en el firmware [6] y en README_EJECUCION.txt.

3.3. Evaluación digital en ESP32

3.3.1. Organización del procesamiento

La evaluación compara un filtro temporal y uno espectral dentro del mismo microcontrolador. Ambos reciben la misma mezcla y mantienen estados independientes. Se ejecutan uno después del otro para simplificar la coordinación, distinguir el costo de cada método y reutilizar parte de la memoria de trabajo; la Figura 3.4 muestra esa secuencia.

La referencia limpia se emplea para calcular las métricas. En el método espectral, las decisiones de atenuación se obtienen únicamente desde la mezcla; después se aplican esas mismas ganancias a la referencia para medir la conservación de la señal útil. Así, conocer la señal limpia no mejora artificialmente la detección del ruido.

3.3.2. Filtro temporal y filtro espectral

El filtro temporal es de respuesta finita al impulso (FIR). Combina cada muestra con las 74 anteriores mediante los 75 coeficientes exportados del modelo y conserva ese historial entre bloques, de forma que procesar la señal por partes no reinicia el filtro.

El filtro espectral analiza ventanas de 1024 muestras con una FFT radix 2 propia, selecciona las componentes que superan el umbral de energía y les aplica una ganancia de 0.1 en amplitud; las frecuencias inferiores a 1600 Hz quedan protegidas. Para reconstruir la señal se aplica la IFFT. La ventana Hann suaviza los bordes y el solapamiento, junto con la compensación de pesos, une los bloques sin introducir cambios de amplitud. El umbral se actualiza en cada ventana [6].

Elemento	Configuración
FIR	75 coeficientes; bloques de 256 muestras; retardo de 37 muestras
Ventana espectral	1024 muestras; Hann simétrica; avance de 512 (50 %)
Selección espectral	Desde 1600 Hz; ancho 100 Hz; ganancia 0.1 en amplitud
Umbral relativo	0.05 del pico regional, con mínimo de 0.001 del pico global
Intervalo común de métricas	96 000 – 37 = 95 963 muestras

Tabla 3.6. Configuración de los filtros y del intervalo de comparación en ESP32.

3.3.3. Alineación y cálculo de métricas

El FIR introduce un retardo conocido de 37 muestras, que se compensa para comparar su salida con la referencia: quedan 95 963 pares alineados. El espectral reconstruye las 96 000 muestras, pero se evalúa sobre las mismas 95 963, de modo que las diferencias provienen del procesamiento y no del intervalo. El firmware valida esa cantidad en cada evaluación.

Se calculan la SNR de entrada y salida, la mejora de SNR, el MSE, el porcentaje de energía útil conservada, la atenuación del ruido y la SNR de distorsión, que compara la referencia

limpia con su versión filtrada. Las energías se acumulan en doble precisión, con las mismas definiciones que el modelo Python [1], [6].

3.3.4. Criterios de diseño y compromisos

Decisión	Motivo	Compromiso asociado
Reutilizar señales y semillas del modelo	Comparar las mismas condiciones en Python y ESP32	Las señales quedan fijadas antes de ejecutar
Guardar los registros en flash	Repetir la prueba sin transmitir muestras	Firmware más grande; cambiar señales exige recompilar
DAC integrado de 8 bits	Montaje simple sin convertidor externo	Resolución de amplitud limitada
Emisión y filtros en secuencia	Coordinación simple; costos separados	La emisión y el filtrado no ocurren a la vez
Bloques y estados acotados	Memoria de trabajo reducida	Conservar historial FIR y solapamiento
Ventanas de 1024 muestras	Separación de frecuencias adecuada	Más memoria y cálculo por ventana
Proteger frecuencias < 1600 Hz	No atenuar los tonos útiles	El ruido en esa banda también queda
Evaluar ambos filtros en 95 963 muestras	Comparación justa y compatible con Python	Métricas limitadas al intervalo común

Tabla 3.7. Decisiones de diseño, justificación y compromisos del emisor y la evaluación.

3.4. Receptor y estrategia de validación

El nodo 1 del proyecto se implementó en una Raspberry Pi Pico (RP2040), programada en C con el SDK de la Pico [11], en proyecto_1_grupal/firmware/receptor [6]. Recibe la señal analógica del emisor por el ADC, la analiza por bloques y publica un resumen por USB cada segundo. La Figura 3.5 muestra la conexión y el procesamiento, y la Figura 3.6 el criterio con el que se decide si la recepción es correcta. Sus métodos se verificaron con pruebas en computadora (Sección 3.4.4).

3.4.1. Conexión y acondicionamiento

El DAC del ESP32 y el ADC de la Pico trabajan en el mismo intervalo de 0 a 3.3 V, por lo que la señal unipolar del emisor entra al ADC sin desplazamiento ni ganancia [10], [12], [13].

Ambas placas comparten GND. Entre GPIO25 y GPIO26 se coloca un filtro RC pasabajos (Tabla 3.8). Como la Pico muestrea a 10 kHz, todo lo que supere los 5 kHz se refleja dentro de la banda; la salida escalonada del DAC a 16 kHz contiene imágenes en 16 kHz ± f, y la imagen de 12 kHz del tono de 4 kHz aparecería en 2 kHz. La retención del DAC y el filtro RC la dejan unos 16 dB por debajo de ese tono, sin cambiar el pico dominante. La resistencia en serie limita además la corriente si una placa queda sin alimentación. El firmware admite también un micrófono MAX4466 alimentado a 3.3 V como entrada alternativa, sin unir ambas salidas.

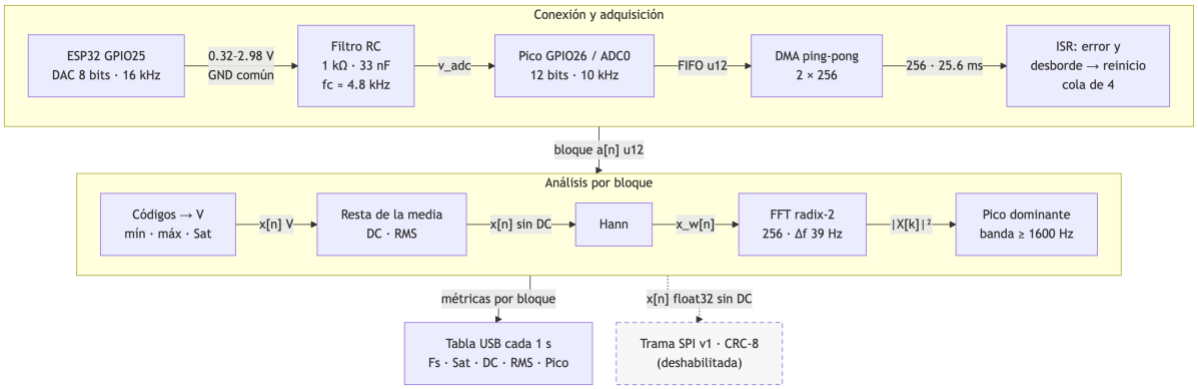


Figura 3.5. Conexión y procesamiento del receptor Pico. Borde discontinuo: deshabilitado en este avance.

Punto o elemento	Valor	Comentario
Salida ESP32 (GPIO25)	≈ 0.32–2.98 V; media 1.65 V	Códigos 25–230; el DAC se deshabilita entre casos
Entrada Pico (GPIO26 / ADC0)	0–3.3 V; 12 bits (0–4095)	Margen ≈ 0.3 V por extremo; Sat cuenta 0 y 4095
Filtro RC de entrada	R = 1 kΩ, C = 33 nF, fc ≈ 4.8 kHz	Atenúa 0.3 dB a 1320 Hz, 2.3 dB a 4 kHz y 8.6 dB a 12 kHz
Referencia	GND común (AGND en la Pico)	Sin ella el nivel DC no es válido

Tabla 3.8. Niveles y elementos de la conexión emisor–receptor.

3.4.2. Adquisición y muestreo

El ADC de la RP2040 deriva su reloj de 48 MHz y el periodo de muestreo se fija con un divisor: 48 MHz / 4800 = 10 kHz [12]. Las conversiones pasan por la FIFO del ADC a dos canales DMA encadenados que llenan alternadamente dos buffers de 256 muestras (25.6

ms cada uno), de modo que la captura continúa mientras se analiza el bloque anterior [11]. La interrupción del DMA solo valida y publica: descarta el bloque si alguna muestra trae el bit de error del ADC, lo copia a una cola de cuatro bloques y, si detecta desborde de la FIFO o un rearme perdido, detiene la captura, cuenta un reinicio y vuelve a empezar sin publicar bloques discontinuos.

Con 10 kHz, la frecuencia de Nyquist de 5 kHz supera la componente más alta de los casos (4 kHz). Los bloques de 256 muestras dan una resolución de 39.06 Hz, suficiente para separar 440, 880 y 1320 Hz por unos 11 bins, con poca memoria y bajo costo de cálculo. El emisor y el receptor no comparten reloj, pero el análisis es por bloques en frecuencia y no necesita sincronizar muestras. En el Taller 2, la versión en CircuitPython no sostuvo una FFT de 1024 puntos ni un muestreo estable [6]; por eso el receptor se escribió en C.

3.4.3. Procesamiento e identificación de frecuencias

Cada bloque recorre la cadena de entrada, ADC, corrección de DC, bloques, Hann, FFT e identificación (Figura 3.5). Los códigos se convierten a voltios con la referencia nominal de 3.3 V y se registran el mínimo, el máximo y las muestras en 0 o 4095 (Sat). La corrección de DC resta la media del bloque, que se conserva como nivel DC, y con el resultado se calcula el RMS de alterna. Sobre una copia se aplican la ventana Hann, necesaria porque los tonos no coinciden con bins enteros (Tabla 3.9) [4], y una FFT radix 2 de 256 puntos [5]. Del espectro se obtienen el pico dominante y la componente más fuerte por encima de 1600 Hz, candidata a ruido.

Cada segundo, la Pico resume por USB todos los bloques del intervalo: nivel (ACTIVIDAD si el RMS máximo alcanza 5 mV), RMS medio y máximo, frecuencia del pico del bloque más intenso, DC, tensión pico a pico, Sat, tasa de muestreo medida con el DMA y contadores de bloques perdidos, errores del ADC y reinicios. A pedido envía además un bloque crudo para graficar su forma de onda en la computadora. Además, cada bloque sin DC se empaqueta en una trama SPI (cabecera, identificador, 256 muestras float32 y CRC-8) diseñada para el DMA del ESP32 como esclavo [14]. A 2 MHz, cada transacción de 1032 bytes tarda unos 4.1 ms, menos que los 25.6 ms de un bloque, de modo que la tasa de comunicación es compatible con el muestreo. La transmisión se habilita al compilar con la opción NODO1_HABILITAR_SPI; en este avance se mantiene deshabilitada porque ambos filtros se evalúan dentro del ESP32.

Componente	f (Hz)	k = f·N/fs	Bin más cercano (Hz)	Casos
Tono útil 1	440	11.26	11 (429.7)	1, 2 y 3
Tono útil 2	880	22.53	23 (898.4)	1, 2 y 3
Tono útil 3	1320	33.79	34 (1328.1)	1, 2 y 3
Interferencia tonal	4000	102.40	102 (3984.4)	1
Banda de ruido	2500–3800	64.0–97.3	—	2
Banda superpuesta	600–1500	15.4–38.4	—	3

Tabla 3.9. Posiciones esperadas en la FFT de 256 muestras a 10 kHz.

3.4.4. Detección de problemas y criterio de decisión

La validación se aplica a las filas de la consola mientras dura cada emisión de 6 s, con la secuencia de la Figura 3.6. Entre un caso y otro el ESP32 deshabilita el DAC y la entrada queda sin excitación, por lo que esas filas no se evalúan. Primero se comprueba la adquisición: la tasa medida debe estar en 10 000 Hz, sin la marca de intervalo inválido, y los contadores de pérdidas, errores y reinicios deben seguir en cero. Una tasa distinta revela un divisor mal configurado; si la captura asumiera 10 kHz con otra tasa real, todos los picos se desplazarían en la misma proporción (por ejemplo, a 16 kHz reales el tono de 440 Hz aparecería cerca de 275 Hz).

Después se revisa el nivel: Sat debe ser 0, porque una muestra en el extremo ya perdió información, y la media debe estar en 1.65 ± 0.17 V (± 5 % de la escala); fuera de ese intervalo hay un problema de GND o una entrada flotante. Con señal, el nivel debe indicar ACTIVIDAD. Por último, el pico dominante debe estar en el bin 11 ± 1 (390.6–468.8 Hz), que corresponde al tono de 440 Hz, el de mayor amplitud en los tres casos. En el caso 3 el ruido entre 600 y 1500 Hz puede llevar algunos bloques a la zona de 880 Hz; un rechazo así indica superposición y no un fallo de conexión. La Tabla 3.10 resume los valores esperados, calculados a partir de los códigos DAC del firmware.

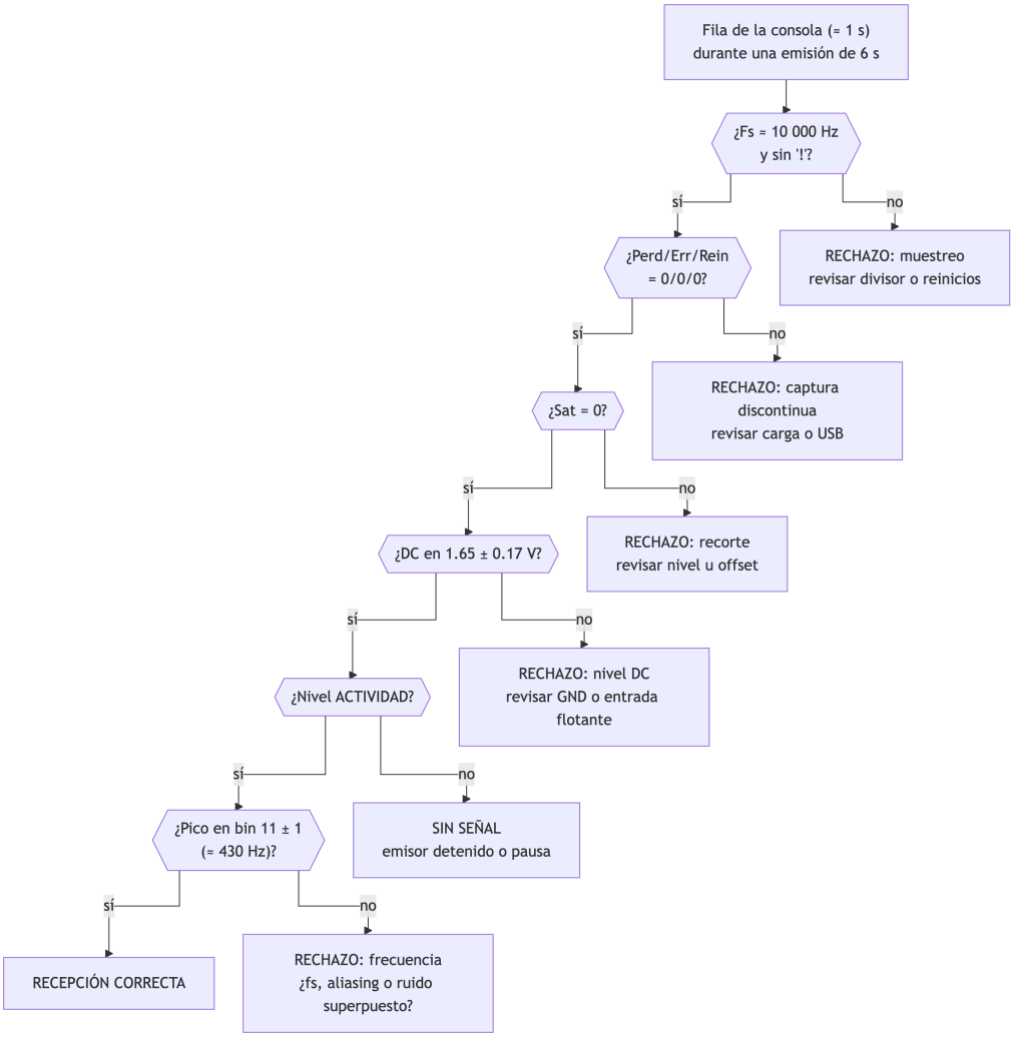


Figura 3.6. Diagrama de decisión del receptor sobre cada fila de la consola.

Caso	Fs (Hz)	DC (V)	Vpp (V)	RMS por bloque (mV)	Pico (Hz)	Sat
caso_1	≈ 10 000	1.65	≤ 2.45	470–635	429.7	0
caso_2	≈ 10 000	1.65	≤ 2.64	330–455	429.7	0
caso_3	≈ 10 000	1.65	≤ 2.60	295–440	429.7 *	0

Tabla 3.10. Valores esperados en la consola del receptor durante cada emisión, sin el filtro RC. * En algunos bloques del caso 3 puede aparecer la zona de 880 Hz.

Los métodos del receptor se verificaron en computadora con las pruebas de su carpeta: la prueba en C compara la FFT con una DFT directa y comprueba la corrección de DC, el RMS,

la ventana, la estimación del pico, el CRC-8 y el formato de la trama; las cuatro pruebas en Python validan el protocolo de captura por USB y la conversión a voltios. Ambas terminaron sin errores el 30 de octubre de 2026.

4. Validación y resultados

4.1. Tabla de pruebas

La Tabla 4.1 reúne las pruebas de este avance con su señal, la frecuencia esperada, el método de comprobación, el criterio de aceptación y el estado de la evidencia. Los criterios se fijaron antes de ejecutar las pruebas, como pide el proyecto [1].

ID	Señal emitida	Frecuencia esperada	Método de comprobación	Criterio de aceptación	Estado
V1	Casos 1–3 por DAC	440/880/1320 Hz (+4000 Hz en caso 1)	Consola: códigos reproducidos y paradas	96 000 reproducidas, 0 paradas, demostración completa	Ejecutado en ESP32
V2	Casos 1–3 hacia la Pico	Pico en bin 11 ± 1 (≈ 430 Hz)	Consola del receptor (Fig. 3.6)	Filas de cada emisión aceptadas (Tabla 3.10)	Implementado en la Pico
V3	Bloques sintéticos en computadora	Tonos de prueba	Pruebas de métodos del receptor (C y Python)	FFT = DFT; DC, RMS, Hann, pico, CRC-8 y trama correctos	Ejecutado en computadora
V4	Entrada a GND o nivel fuera de rango	—	Columnas Sat y DC	Rechazo por recorte o por nivel DC	Implementado en la Pico
V5	Divisor del ADC mal configurado	Picos desplazados	Columnas Fs y Pico	Rechazo por muestreo	Implementado en la Pico
V6	Casos 1–3, FIR y espectral	—	ESP32 frente a Python con las mismas muestras	$\|\Delta$ métricas$\| \leq 0.03$ dB; MSE ± 10^{-7}	Ejecutado en ESP32
V7	Casos 1–3	—	esp_timer por bloque	Máximo < 16 ms (FIR) y < 32 ms (espectral)	Ejecutado en ESP32
V8	Casos 1–3 en Python	—	prototipo.py desde cero y pruebas automáticas	Métricas iguales a la Tarea 1; pruebas aprobadas	Ejecutado

Tabla 4.1. Tabla de validación del avance.

4.2. Resultados del emisor y de la evaluación en ESP32

En la ejecución de consola del 30 de octubre de 2026 se completaron tres reproducciones y seis evaluaciones. El controlador reportó 96 000 códigos reproducidos y cero paradas inesperadas en cada caso; ambos filtros evaluaron 95 963 muestras y la secuencia terminó con el mensaje DEMOSTRACION COMPLETADA. Estos datos acreditan la emisión por el DAC y la ejecución de los cálculos digitales en la placa.

El código cuenta además con 117 pruebas automáticas, repetidas durante la integración el 30 de octubre: 94 del modelo Python, 16 del núcleo C de los filtros compilado en computadora y 7 del cierre del DAC con eventos simulados. Son comprobaciones distintas de la ejecución en la placa.

```
================ RESUMEN FINAL ================
Comparacion comun: 95963 muestras evaluadas por metodo.

CALIDAD DIGITAL: SNR, mejora, atenuacion y distorsion en dB; util en %
| Caso   | Metodo    | SNR ent  | SNR sal  | Mejora   | MSE         | Util %    | Aten dB  | Dist dB  |
|--------|-----------|----------|----------|----------|-------------|-----------|----------|----------|
| caso_1 | FIR       |  10.0017 |  69.0489 |  59.0472 |  1.655e-08  |  100.0013 |  59.1784 |  84.2617 |
| caso_1 | ESPECTRAL |  10.0017 |  29.9760 |  19.9743 |  1.337e-04  |  100.0000 |  19.9743 | 136.7320 |
| caso_2 | FIR       |   5.0017 |  65.1396 |  60.1379 |  1.507e-08  |  100.0013 |  60.1926 |  84.2605 |
| caso_2 | ESPECTRAL |   5.0017 |  23.3486 |  18.3468 |  2.276e-04  |  100.0000 |  18.3468 | 133.4216 |
| caso_3 | FIR       |   0.0005 |   0.0034 |   0.0029 |  2.239e-02  |  100.0013 |   0.0029 |  84.2614 |
| caso_3 | ESPECTRAL |   0.0005 |   0.0027 |   0.0022 |  2.239e-02  |  100.0000 |   0.0022 | 108.0627 |

TIEMPOS: media, maximo y presupuesto por bloque en ms; calculo y total en s
| Caso   | Metodo    | Evaluadas | Media ms | Max ms   | Pres ms | Cumple | Calculo s | Total s  |
|--------|-----------|-----------|----------|----------|---------|--------|-----------|----------|
| caso_1 | FIR       |     95963 |    8.740 |    8.828 |  16.000 | SI     |     3.277 |    3.758 |
| caso_1 | ESPECTRAL |     95963 |   18.658 |   18.863 |  32.000 | SI     |     3.545 |    3.833 |
| caso_2 | FIR       |     95963 |    8.738 |    8.828 |  16.000 | SI     |     3.277 |    3.753 |
| caso_2 | ESPECTRAL |     95963 |   18.703 |   18.945 |  32.000 | SI     |     3.554 |    3.834 |
| caso_3 | FIR       |     95963 |    8.711 |    8.803 |  16.000 | SI     |     3.267 |    3.753 |
| caso_3 | ESPECTRAL |     95963 |   18.595 |   18.838 |  32.000 | SI     |     3.533 |    3.834 |
```

Figura 4.1. Resumen final de la consola de la ESP32 (ejecución del 30/10/2026).

Caso	Filtro	SNR entrada (dB)	SNR salida (dB)	Mejora (dB)	MSE
caso_1	FIR	10.0017	69.0489	59.0472	1.655e-08
caso_1	Espectral	10.0017	29.9760	19.9743	1.337e-04
caso_2	FIR	5.0017	65.1396	60.1379	1.507e-08
caso_2	Espectral	5.0017	23.3486	18.3468	2.276e-04
caso_3	FIR	0.0005	0.0034	0.0029	2.239e-02
caso_3	Espectral	0.0005	0.0027	0.0022	2.239e-02

Tabla 4.2. SNR y MSE medidos digitalmente en ESP32 sobre 95 963 muestras por evaluación.

Caso	Filtro	Energía útil (%)	Atenuación del ruido (dB)	SNR de distorsión (dB)
caso_1	FIR	100.0013	59.1784	84.2617
caso_1	Espectral	100.0000	19.9743	136.7320
caso_2	FIR	100.0013	60.1926	84.2605
caso_2	Espectral	100.0000	18.3468	133.4216
caso_3	FIR	100.0013	0.0029	84.2614
caso_3	Espectral	100.0000	0.0022	108.0627

Tabla 4.3. Conservación de energía útil, atenuación del ruido y distorsión digital en ESP32.

El valor de 100.0013 % del FIR indica una variación pequeña de ganancia en la banda de paso; no significa una recuperación perfecta. Del mismo modo, los valores altos de SNR de distorsión describen diferencias digitales muy pequeñas y no representan la resolución del DAC de 8 bits.

Los tiempos se midieron con ESP Timer, de resolución de microsegundos, con el procesador a 160 MHz [15]. El tiempo por bloque incluye la lectura de datos, el filtrado de la mezcla y de la referencia y la acumulación de métricas, y excluye las pausas voluntarias para FreeRTOS. El presupuesto es la duración del bloque a 16 kHz: 16 ms para 256 muestras (FIR) y 32 ms para el avance de 512 muestras (espectral). Todos los máximos quedaron

dentro del presupuesto (Tabla 4.4). El banco de filtros reportó 53 272 bytes de memoria de trabajo compartida, reutilizada por ambos métodos.

Caso	Filtro	Media (ms)	Máximo (ms)	Presupuesto (ms)	Cumple	Cálculo (s)	Total (s)
caso_1	FIR	8.740	8.828	16	Sí	3.277	3.758
caso_1	Espectral	18.658	18.863	32	Sí	3.545	3.833
caso_2	FIR	8.738	8.828	16	Sí	3.277	3.753
caso_2	Espectral	18.703	18.945	32	Sí	3.554	3.834
caso_3	FIR	8.711	8.803	16	Sí	3.267	3.753
caso_3	Espectral	18.595	18.838	32	Sí	3.533	3.834

Tabla 4.4. Tiempos por bloque y acumulados en ESP32 (valores de la Figura 4.1).

Esta comparación describe el costo por bloque de la evaluación secuencial; no equivale a la latencia de una cadena completa de adquisición, filtrado y salida.

4.3. Comparación entre simulación y ESP32

La Tabla 4.5 compara las métricas del prototipo Python (simulación en float64) con las calculadas en la ESP32 (float32) sobre las mismas muestras: ambas columnas son cálculos digitales, en plataformas distintas.

Caso	Filtro	SNR sal. Python	SNR sal. ESP32	|Dif.| (dB)	MSE Python	MSE ESP32	Dist. Python	Dist. ESP32
caso_1	FIR	69.0488	69.0489	0.0001	1.655e-08	1.655e-08	84.26	84.26
caso_1	Espectral	29.9760	29.9760	0.0000	1.337e-04	1.337e-04	154.84	136.73
caso_2	FIR	65.1397	65.1396	0.0001	1.507e-08	1.507e-08	84.26	84.26
caso_2	Espectral	23.3486	23.3486	0.0000	2.276e-04	2.276e-04	135.97	133.42
caso_3	FIR	0.0034	0.0034	0.0000	2.239e-02	2.239e-02	84.26	84.26
caso_3	Espectral	0.0027	0.0027	0.0000	2.239e-02	2.239e-02	108.07	108.06

Tabla 4.5. Simulación (Python) frente a ejecución digital (ESP32). SNR y distorsión en dB.

La SNR de salida coincide en ambos filtros con diferencias de hasta 0.0001 dB y el MSE coincide en sus tres cifras significativas, dentro de los criterios de V6. La única diferencia visible es la SNR de distorsión del espectral en los casos 1 y 2 (154.84 frente a 136.73 dB y 135.97 frente a 133.42 dB). Esos valores corresponden a errores relativos cercanos a 10^{-7}, el límite de precisión de float32, por lo que la diferencia se debe al formato numérico y no al algoritmo.

4.4. Resumen de la evidencia

Elemento	Implementación	Evidencia
Prototipo Python	prototipo.py sobre los módulos de la Tarea 1	Ejecución completa; reproduce las métricas de la Tarea 1 (Tabla 3.3)
Emisor ESP32 (DAC)	Firmware C con ESP-IDF 5.5.5	Tres emisiones completas en consola (Figura 4.1)
Filtros en ESP32	FIR y espectral en C	Métricas y tiempos en consola; 16 pruebas del núcleo C
Receptor Pico	Firmware C con el SDK de la Pico	Pruebas de sus métodos en C y Python (V3)
Enlace entre nodos	Trama SPI con CRC-8 definida en la Pico	Prueba del formato y del CRC-8 (V3)

Tabla 4.6. Implementación y evidencia de cada elemento del avance.

5. Discusión

Los casos 1 y 2 muestran que separar señal y ruido en frecuencia facilita la reducción de interferencias. El FIR obtuvo mejoras de SNR de unos 59.05 y 60.14 dB, mientras que el espectral alcanzó 19.97 y 18.35 dB. La diferencia se explica por el diseño: el FIR atenúa al menos 60 dB toda la banda de rechazo, mientras que el espectral aplica una ganancia de 0.1 (–20 dB) a las componentes seleccionadas. En el caso 3 las mejoras son inferiores a 0.003 dB, porque el ruido entre 600 y 1500 Hz comparte la región de los tonos útiles: proteger la señal útil también conserva el ruido superpuesto. La SNR, el MSE y la distorsión permiten distinguir entre eliminar ruido y conservar la información útil.

La coincidencia entre Python y ESP32, con diferencias de hasta 0.0001 dB en la SNR, indica que el firmware implementa los mismos algoritmos que el modelo. Los tiempos máximos por

bloque, de 8.8 ms para el FIR y 18.9 ms para el espectral, dejan margen respecto de 16 y 32 ms. Esos tiempos caracterizan la evaluación secuencial de cada método.

El receptor reutiliza la ventana Hann y la FFT del modelo, con bloques de 256 muestras a 10 kHz, y su consola reúne las comprobaciones de la Figura 3.6: tasa de muestreo, continuidad, saturación, nivel DC, actividad y pico dominante. Las dos primeras detectan fallos de configuración o de captura, y las dos siguientes, fallos de conexión. La consola informa el pico dominante, que en los tres casos corresponde al tono de 440 Hz; la componente más fuerte sobre 1600 Hz, cercana a 3984 Hz en el caso 1, se calcula internamente como estimación del ruido.

El diseño tiene cuatro limitaciones conocidas. El DAC de 8 bits limita la resolución de la señal física y sus imágenes deben atenuarse con el filtro RC antes del ADC. La consola del receptor resume cada segundo el bloque más intenso, por lo que informa el pico dominante y no cada tono por separado. Los dos filtros se evalúan en secuencia dentro del ESP32, sobre las mismas muestras del modelo. Además, los relojes independientes del ESP32 y de la Pico introducen un pequeño desajuste de frecuencia, que la tolerancia de ±1 bin (39 Hz) absorbe.

En conjunto, el avance cubre los tres elementos solicitados: el sistema completo en Python con las frecuencias de trabajo, el emisor en el ESP32 con frecuencias establecidas y la estrategia de validación del receptor, implementada en la Raspberry Pi Pico.

6. Referencias

[1] Instituto Tecnológico de Costa Rica, "Proyecto Grupal 1: Diseño de un sistema distribuido de reducción adaptativa de ruido en tiempo real," CE 1110 Análisis de Señales Mixtas, Cartago, Costa Rica, sep. 2026.

[2] Instituto Tecnológico de Costa Rica, "Sesión asincrónica semana 12: FFT y sistemas de modulación," CE 1110 Análisis de Señales Mixtas, Cartago, Costa Rica, oct. 2026.

[3] A. V. Oppenheim y R. W. Schafer, Discrete-Time Signal Processing, 3.ª ed. Upper Saddle River, NJ, EE. UU.: Pearson, 2010.

[4] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," Proceedings of the IEEE, vol. 66, no. 1, pp. 51–83, ene. 1978.

[5] J. W. Cooley y J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," Mathematics of Computation, vol. 19, no. 90, pp. 297–301, 1965.

[6] N. Florez, T. Cajiao, S. Robles y S. Boza, "tcajiao_asm_2026_s2: proyecto_1_grupal" (prototipo.py, tarea1_grupo1, firmware/esp32, firmware/receptor y docs/avance2, rama development),

repositorio GitHub, 2026. [En línea]. Disponible: https://github.com/TamaraCM06/tcajiao_asm_2026_s2

[7] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020.

[8] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, pp. 261–272, 2020.

[9] Espressif Systems, "Partition Tables," ESP-IDF Programming Guide v5.5, ESP32. [En línea]. Disponible: https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32/api-guides/partition-tables.html

[10] Espressif Systems, "Digital To Analog Converter (DAC)," ESP-IDF Programming Guide v5.5, ESP32. [En línea]. Disponible: https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32/api-reference/peripherals/dac.html

[11] Raspberry Pi Ltd., "Raspberry Pi Pico-series C/C++ SDK." [En línea]. Disponible: https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf

[12] Raspberry Pi Ltd., "RP2040 Datasheet: A microcontroller by Raspberry Pi," sec. 4.9 (ADC). [En línea]. Disponible: https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf

[13] Raspberry Pi Ltd., "Raspberry Pi Pico Datasheet." [En línea]. Disponible: https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf

[14] Espressif Systems, "SPI Slave Driver," ESP-IDF Programming Guide v5.5, ESP32. [En línea]. Disponible: https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32/api-reference/peripherals/spi_slave.html

[15] Espressif Systems, "ESP Timer (High Resolution Timer)," ESP-IDF Programming Guide v5.5, ESP32. [En línea]. Disponible: https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32/api-reference/system/esp_timer.html